

README / Setup Guide

Overview

This expense management application is designed to streamline the process of submitting, approving, and reporting on employee expenses within an organization. It provides a user-friendly interface for employees to submit their expenses and for managers to approve or reject them. The application ensures that all expense data is securely stored and easily accessible for auditing and reporting purposes.

Key Features

- **Expense Submission:** Employees can submit their expense details through a simple and intuitive interface.
- **Approval Workflow:** Managers can approve or reject submitted expenses, ensuring that all expenses are properly vetted.
- **Reporting:** Users can view detailed reports of submitted and approved expenses, aiding in financial oversight and compliance.

Prerequisites

- **Java 11+:** Ensure you have Java 11 or higher installed.
- **PostgreSQL:** A PostgreSQL database should be installed and running.
- **Node.js 16+:** Required for frontend dependencies.
- **Git:** For cloning the repository.
- **Docker** (optional): For running the application in a containerized environment.

Project Structure

The repository is structured as follows:

- **backend/:** Contains the Spring Boot backend responsible for handling business logic, API endpoints, and database interactions.
- **frontend/:** Houses the React frontend responsible for the user interface and interaction with the backend APIs.
- **shared/:** Contains shared utilities and configurations if any.
- **docs/:** Location for project documentation.

Installation & Setup

Backend Setup

1. **Clone the repository:** `git clone <url>`
2. **Navigate to the project:** `cd expense-management-app`
3. **Create virtual environment:** `python -m venv venv`
4. **Activate:** `source venv/bin/activate` (Linux/Mac) or `venv\Scripts\activate` (Windows)
5. **Install dependencies:** `pip install -r backend/requirements.txt`
6. **Configure PostgreSQL:** Ensure your PostgreSQL database is running and accessible. Update the `application.properties` file with your database credentials.

Frontend Setup

1. **Navigate to frontend:** `cd frontend`
2. **Install dependencies:** `npm install`
3. **Build/prepare assets:** `npm run build`

Configuration

Required Environment Variables

- **SPRING_DATASOURCE_URL:** Controls the database connection URL, format: `jdbc:postgresql://localhost:5432/expensedb`, example value: `jdbc:postgresql://localhost:5432/expensedb`
- **SPRING_DATASOURCE_USERNAME:** Controls the database username, format: `username`, example value: `postgres`
- **SPRING_DATASOURCE_PASSWORD:** Controls the database password, format: `password`, example value: `password`

Optional Environment Variables

- **JWT_SECRET** (default: `secret`): Controls the JWT secret key used for authentication, effect if changed: new secret key will be required for authentication

Running the Application

Starting the Backend

1. **Ensure virtual environment is activated**
2. **Set environment variables:** `export SPRING_DATASOURCE_URL=jdbc:postgresql://localhost:5432/expensedb`
3. **Start server:** `cd backend && mvn spring-boot:run`
4. **Verify:** Backend is running on `http://localhost:8080`

Starting the Frontend

1. **Navigate to frontend directory:** `cd frontend`
2. **Start dev server:** `npm run dev`
3. **Open browser:** `http://localhost:3000`
4. **Verify:** UI loads and connects to backend

Verification Checklist

- [] Backend health check responds at GET `/api/health` with 200 OK
- [] Frontend loads without console errors
- [] Basic workflow completes end-to-end (e.g., submit and approve an expense)
- [] API endpoints respond with expected data format

Troubleshooting

Backend Issues

- **Port already in use:** Change PORT env var or kill process on port 8080
- **Module not found:** Run `pip install -r backend/requirements.txt` again
- **Database connection error:** Verify connection string in `application.properties`, check service is running
- **Authentication error:** Verify JWT secret in `application.properties` is correct

Frontend Issues

- **Blank page or won't load:** Check browser console for errors, ensure backend is running
- **API connection error:** Verify backend URL in config, check CORS settings
- **Module resolution error:** Delete `node_modules` and run `npm install` again
- **Build fails:** Ensure Node.js version matches requirements, clear cache: `npm cache clean --force`

Common Workflows

Submit Expense

1. **Login as an employee**
2. **Navigate to the expense submission page**
3. **Fill in the expense details and submit**
4. **Verify:** Expense is submitted successfully and appears in the list

Approve Expense

1. **Login as a manager**
2. **Navigate to the expense approval page**
3. **Select an expense to approve**
4. **Approve the expense**
5. **Verify:** Expense status changes to APPROVED

Deployment Notes

- **Secrets Management:** Ensure sensitive information like database credentials and JWT secrets are stored securely, e.g., using environment variables or a secrets manager.
- **Scaling:** Consider using a load balancer and multiple instances of the backend for high availability.

Getting Help

- Check logs: Backend logs in console, frontend logs in browser DevTools
- Review error message and stack trace for clues
- Check environment variables are set correctly
- Refer to the API Documentation and User Guide for more details